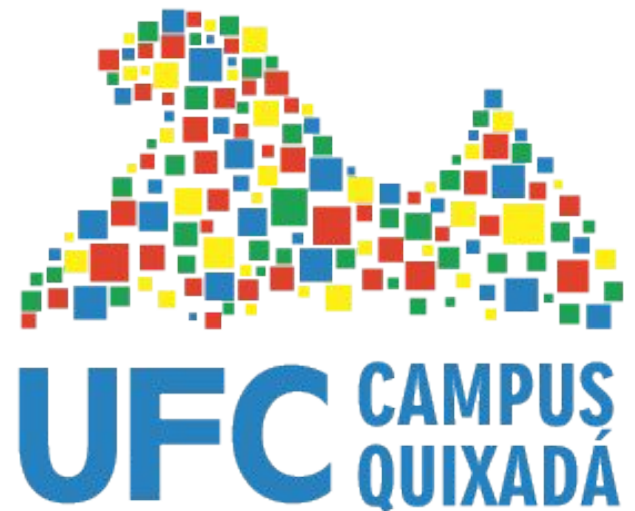


Arquitetura de Software

Atributos de Qualidade

Sidarta Carvalho



Objetivos

- Como expressar as qualidades de um sistema?
- Como alcançar as qualidades desejadas?
- Como definir ações de projeto para garantir as características de qualidade?

Atributo de qualidade

- Um atributo de qualidade (*Quality Attribute* -QA) pode ter suas propriedades medidas ou testadas e podem ser usadas para indicar quão bem o sistema satisfaz as necessidades dos *stakeholders*.
- O atributo de qualidade deve permitir a medição de quão bem uma determinada propriedade satisfaz um determinado *stakeholder*.

Requisitos do sistema

- Os requisitos de um sistema podem ser representados de diversas formas:
 - Requisitos textuais
 - Exemplo: Documento de requisitos
 - Mockups
 - Exemplo: desenhos de sites
 - Protótipos (estágio + avançado de mockup)
 - Reaproveitamento/cópia de sistemas existentes
 - Casos de uso
 - Estórias de usuário

Requisitos do sistema

- Requisitos funcionais
 - Este tipo de requisitos define o que o sistema deve fazer e como deve se comportar.
 - Além disso, também define como o sistema deve reagir a um estímulo
 - Exemplos:
 - O sistema deve permitir a inserção de usuário na base de dados
 - O sistema deve permitir o relacionamento de dados dos usuários com os dados de outros sistemas
 - O sistema deve esconder falhas de sistema do usuário final

Requisitos do sistema

- Requisitos de atributos de qualidade
 - Este tipo de requisito são qualificações dos requisitos funcionais ou de todo o sistema.
 - Qualificações dos requisitos:
 - Quanto rápido uma função pode ser executada
 - Quanto bem o sistema consegue se recuperar após uma falha
 - Qualificação do sistema:
 - Qual o tempo para implantar o sistema no cliente

Requisitos do sistema

- Requisitos restritivos
 - Uma restrição é uma decisão de projeto do sistema que não pode ser ajustada ou modificada, ou seja, deve ser respeitada pelo sistema.
 - Uma restrição é uma decisão que já foi feita para você!
 - Exemplos:
 - Utilizar a linguagem de programação Java
 - Reutilizar o módulo de criptografia X
 - Uso de desenvolvimento orientado a serviços

Atributos de qualidade

- Quando o usuário pressionar o botão verde, um diálogo com opções deve aparecer
 - Quais atributos de qualidade podemos ter aqui?



Atributos de qualidade

- Quando o usuário pressionar o botão verde, um diálogo com opções deve aparecer
 - Atributo de qualidade envolvendo **desempenho**:
 - Quão **rápido** o menu de opções aparece após o clique?
 - Disponibilidade
 - Qual a porcentagem de cliques onde o menu de opções não apareceu?
 - Usabilidade
 - Quão fácil é para aprender a clicar e visualizar o menu de opções?

Atributos de qualidade

- Como testar um atributo de qualidade?
 - É possível dizer quão modificável um sistema é?
- Como identificar a qual atributo de qualidade um determinado conceito pertence?
 - Disponibilidade vs Confiabilidade vs Segurança
- Quais conceitos são corretos? Os usados para descrever confiabilidade ou disponibilidade?
 - Cada categoria cria o seu vocabulário

Atributos de qualidade

- Para amenizar estes questionamentos, podemos fazer uso de:
 - Cenários dos atributos de qualidade
 - Definir conceitos únicos para todos (ou maioria) dos atributos de qualidade
- Cenários para disponibilidade
 - Número de falhas e suas consequências
 - Uma falha é quando o sistema pára de responder ao usuário e deixa de entregar o serviço
 - Quando uma falha é observável para os usuários

Especificação de atributos de qualidade

- Um atributo de qualidade deve ser **não ambíguo** e **testável**!
- Fazer uso de cenários para descrever os atributos de qualidade

Especificação de atributos de qualidade

- O que um cenário deve conter?
 - Estímulo
 - Fonte do estímulo
 - Resposta ao estímulo
 - Medição da resposta
 - Ambiente
 - Artefato

Especificação de atributos de qualidade

- O grande problema ao especificar atributos de qualidade é a ambiguidade.
- Quando um stakeholder diz: "O sistema do PIX tem que ser altamente disponível e não pode falhar", o que isso significa na prática para quem vai programar?
- A motivação para usar a estrutura de cenários (Fonte, Estímulo, Ambiente, Artefato, Resposta e Medição) é transformar adjetivos subjetivos em métricas objetivas e testáveis.

Especificação de atributos de qualidade

- Fonte do estímulo
 - Qualquer entidade (humano, computador ou outra) que gera o estímulo
- Estímulo
 - Estímulo é um evento que exige uma resposta ao chegar no sistema computadorizado
- Ambiente
 - O estímulo acontece diante de algumas condições
 - O sistema pode estar sobrecarregado ou em operação normal
 - O que é normal?

Especificação de atributos de qualidade

- Artefato
 - Entidade que é estimulada
 - Pode ser o sistema completo, uma parte dele ou algum módulo em específico
- Resposta
 - Ações que são disparadas após a chegada do estímulo
- Medição da resposta
 - Medição da resposta recebida, baseada em algum aspecto que permita o **teste**

Exemplo

Elemento do Cenário	Definição Teórica (O que é)	Aplicação Prática (Exemplo do PIX)
Fonte do Estímulo	Quem ou o que gera a ação.	O cliente (usuário final) através do aplicativo do banco no smartphone.
Estímulo	O evento que exige uma resposta.	O envio de uma solicitação de transferência PIX.
Ambiente	As condições do sistema no momento.	O sistema operando sob carga normal, mas o servidor (worker) principal do Orquestrador sofre uma falha de hardware fatal no exato momento em que recebe a requisição.
Artefato	Qual parte do sistema é afetada.	O serviço distribuído de processamento do PIX (Gateway, Filas de Mensageria e Orquestrador).
Resposta	Como a arquitetura reage.	O Gateway coloca a mensagem na Fila (Kafka/SQS). O worker principal cai, mas a Fila retém a mensagem por <i>timeout</i> . Outro worker saudável em uma Zona de Disponibilidade (AZ) diferente consome a mensagem e executa o débito/crédito de forma atômica no banco de dados.
Medição da Resposta	O critério de sucesso (testável).	Zero transações perdidas (nenhum dinheiro debitado sem ser creditado) e a transação completa é confirmada para o usuário em menos de 5 segundos , apesar da falha na infraestrutura.

Atributos de Qualidade

Categorias dos Atributos de Qualidade



Requisitos de qualidade

ISO/IEC 25010:2023 - Atributos de Qualidade

1. Adequação Funcional

- Completude Funcional
- Correção Funcional
- Pertinência Funcional

2. Eficiência de Desempenho

- Comportamento Temporal
- Utilização de Recursos
- Capacidade

3. Compatibilidade

- Coexistência
- Interoperabilidade

4. Capacidade de Interação

- Reconhecimento da Adequação
- Apreensibilidade
- Operabilidade
- Proteção contra Erros do Usuário
- Estética da Interface
- Acessibilidade

5. Confiabilidade

- Maturidade
- Disponibilidade
- Tolerância a Falhas
- Recuperabilidade

6. Segurança

- Confidencialidade
- Integridade
- Não-repúdio
- Responsabilização
- Autenticidade

7. Manutenibilidade

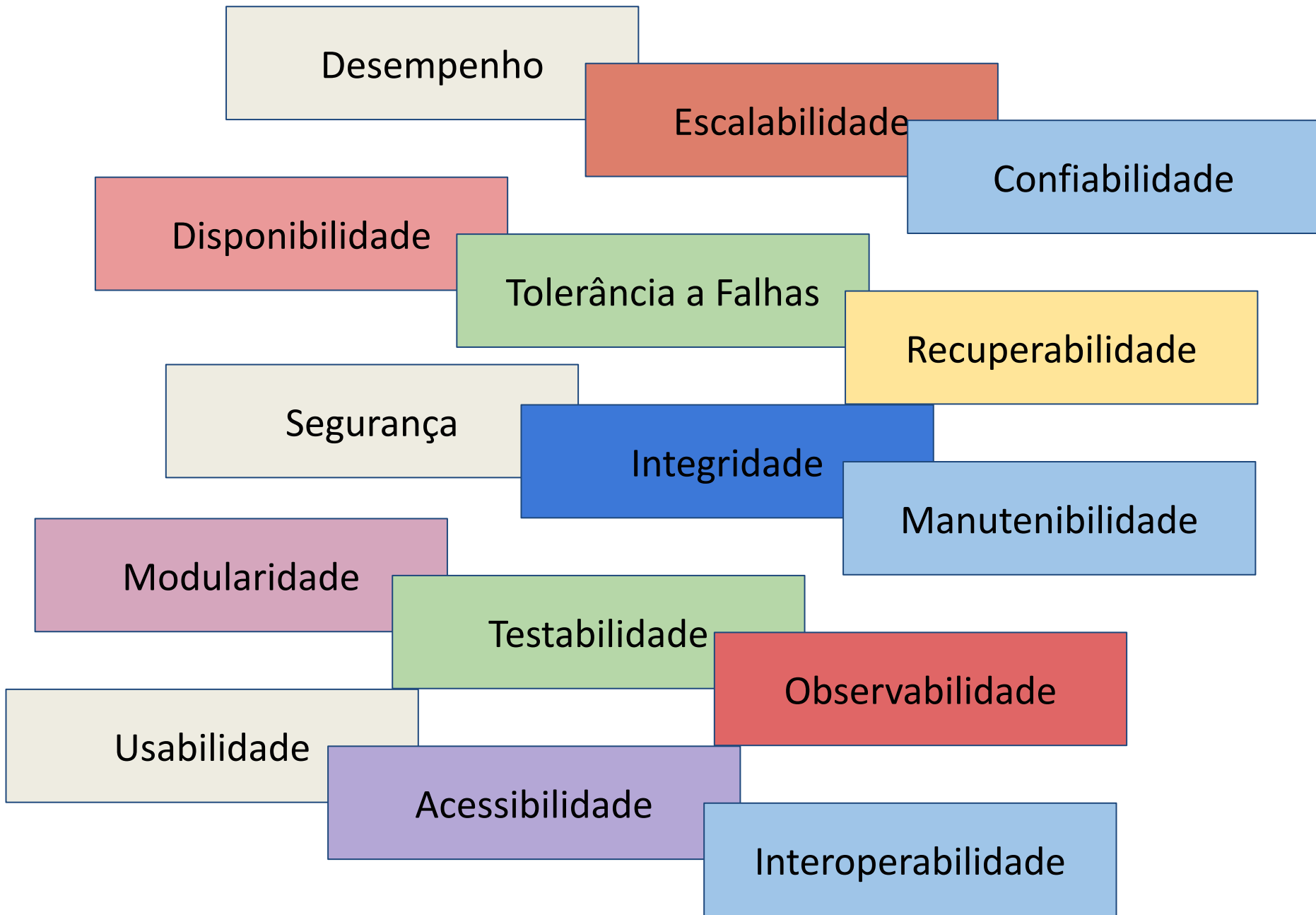
- Modularidade
- Reusabilidade
- Analisabilidade
- Modificabilidade
- Testabilidade

8. Flexibilidade

- Adaptabilidade
- Escalabilidade
- Instalabilidade
- Capacidade de Substituição

9. Proteção (Safety)

- Mitigação de Risco Operacional
- Mitigação de Risco à Saúde/Vida
- Mitigação de Risco Ambiental
- Mitigação de Risco Financeiro



Desempenho

Desempenho

- Desempenho de um sistema em relação à quantidade de recursos que ele utiliza sob determinadas condições.
- Sub-características críticas
 - Comportamento de tempo (latência de resposta e taxa de transferência)
 - Utilização de recursos (CPU, memória, armazenamento, rede)
 - Capacidade (os limites máximos que o sistema pode suportar).
- Em essência, um sistema com alto desempenho responde rapidamente às solicitações do usuário enquanto consome recursos de forma otimizada.

Desempenho

- Nem sempre ser “somente” rápido é ter um bom desempenho, também é preciso otimizar os algoritmos internamente para conseguir entregar mais com menor uso de recursos.
- Por exemplo, a Netflix possui um sistema de codificação de vídeo que aplica uma taxa maior de frames para cenas dinâmicas e uma taxa menor para cenas estáticas, isso permite economizar CPU, memória e dados trafegados pela rede sem perder a qualidade (objetivo final do negócio!)
- Se o negócio também não for levado em consideração, pode chegar um momento que não é mais possível “otimizar” de forma geral o sistema.

Escalabilidade

Escalabilidade

- A escalabilidade é a capacidade de um sistema de software de lidar com uma carga de trabalho crescente de forma eficiente.
- Isso pode ser alcançado de duas maneiras principais:
 - Escalabilidade vertical
 - Aumentar os recursos de uma única máquina, como CPU ou RAM)
 - Escalabilidade horizontal
 - Adicionar mais máquinas ao sistema.
- Para aplicações web de grande escala, a escalabilidade horizontal é a abordagem predominante, pois oferece maior resiliência e capacidade virtualmente ilimitada.

Escalabilidade

- O Instagram, uma plataforma que atende a bilhões de usuários e processa dezenas de milhões de uploads de fotos e vídeos diariamente.
- Sua arquitetura é projetada para otimizar a experiência mais comum e crítica: a **visualização rápida do feed** de notícias.

Escalabilidade

- A estratégia central para a escalabilidade do feed do Instagram é conhecida como "fan-out on write" (distribuição na escrita).
- Quando um usuário publica uma nova foto, em vez de esperar que seus seguidores solicitem o feed para então calcular o que mostrar, o sistema proativamente insere a nova postagem nas "caixas de entrada" de feed de cada um de seus seguidores.
- Quando um seguidor abre o aplicativo, o sistema simplesmente precisa ler essa caixa de entrada pré-calculada, o que é uma operação extremamente rápida.

Escalabilidade

- Esta abordagem revela um trade-off fundamental na arquitetura de sistemas escaláveis.
- Calcular o feed em tempo real ("fan-out on read") seria computacionalmente intensivo e lento, pois exigiria consultar as postagens de todas as contas que um usuário segue a cada atualização.
- Ao optar pelo "fan-out on write", o Instagram otimiza para leituras rápidas, melhorando o desempenho percebido pelo usuário.
- No entanto, isso cria um desafio de escalabilidade de escrita. Se um usuário com 50 milhões de seguidores posta uma foto, o sistema precisa realizar 50 milhões de operações de escrita.

Escalabilidade

- Para gerenciar isso e outros desafios de escala, a arquitetura do Instagram emprega várias técnicas:
 - Arquitetura de Microsserviços: O sistema é decomposto em serviços menores e independentes (serviço de usuário, serviço de mídia, serviço de feed, serviço de notificações), cada um podendo ser escalado de forma autônoma.
 - Armazenamento de Objetos: Fotos e vídeos são armazenados em sistemas de armazenamento de objetos altamente escaláveis, como o Amazon S3, que podem lidar com petabytes de dados.

Escalabilidade

- Para gerenciar isso e outros desafios de escala, a arquitetura do Instagram emprega várias técnicas:
 - Sharding de Banco de Dados: Os bancos de dados que armazenam metadados (como informações de usuários e posts) são particionados horizontalmente (sharded), geralmente pelo `user_id`, para distribuir a carga de leitura e escrita por múltiplos servidores de banco de dados.
 - Cache Intensivo: Camadas de cache, como Redis, são usadas extensivamente para armazenar dados acessados com frequência, como perfis de usuários e feeds, reduzindo a carga sobre os bancos de dados primários.

Confiabilidade

Confiabilidade

Capacidade de um sistema de executar suas funções sem falhas.

Enquanto a disponibilidade foca em minimizar o tempo de inatividade, a confiabilidade foca em **minimizar a frequência das falhas**.

É um atributo primordial em sistemas onde as consequências de uma falha são graves.

Em setores como o financeiro, a confiabilidade é inegociável. A falha em processar uma transação corretamente pode resultar em perdas financeiras significativas e danos à reputação.

Outros sistemas que exigem alta confiabilidade são os que podem pôr vidas em risco, como os embarcados de automóveis, aviões, UTIs, etc.

Outro exemplo prático de confiabilidade é o salvamento automático do Google Docs e similares.

Você está escrevendo um documento muito importante. Um relatório, um trabalho de faculdade, um contrato, um TCC...

No meio do processo, acontece uma falha inesperada:
A energia da sua casa acaba e o computador desliga.
O navegador de internet trava e fecha sozinho.
Sua conexão com a internet cai.

Confiabilidade

A confiabilidade é a garantia fundamental de que seu trabalho **não será perdido**.

Não se trata da velocidade do sistema, da beleza da interface ou de quantas funcionalidades ele tem.

Trata-se da simples e absoluta confiança de que, mesmo que o mundo ao redor do software falhe (seu computador, sua internet), o sistema cumprirá sua promessa de proteger sua informação.

Disponibilidade

Disponibilidade

A disponibilidade é a probabilidade de um sistema estar operacional e acessível quando necessário.

É comumente expressa como uma porcentagem de tempo de atividade (uptime) ao longo de um período.

Ex: 99,9% ou "três noves" de disponibilidade.

Alta disponibilidade é alcançada através da eliminação de pontos únicos de falha.

Disponibilidade

Provedores de nuvem como a Amazon Web Services (AWS) construíram sua infraestrutura global com a alta disponibilidade como um princípio de design fundamental.

O conceito central para alcançar alta disponibilidade na AWS é a sua estrutura de **Regiões e Zonas de Disponibilidade** (Availability Zones - AZs).

Disponibilidade

Uma **Região** é uma área geográfica física no mundo onde a AWS possui múltiplos data centers. Cada Região consiste em múltiplas Zonas de Disponibilidade.

Uma **AZ** é um ou mais data centers discretos, fisicamente separados e isolados, com energia, refrigeração e rede redundantes.

As AZs dentro de uma Região são conectadas por redes de alta largura de banda e baixa latência.

Disponibilidade

Imagine que você está construindo um aplicativo de e-commerce que precisa estar sempre online e ser rápido para seus clientes. Você decide hospedar na AWS.

Escolha da Região

Primeiro, você escolhe uma Região que esteja geograficamente próxima da maioria dos seus clientes para minimizar a latência.

Digamos que seus clientes estão no Brasil, então você escolhe a Região "sa-east-1" (São Paulo).

Essa Região tem toda a infraestrutura da AWS, como servidores, redes e armazenamento, isolada de outras Regiões como "us-east-1" (Virgínia do Norte) ou "eu-west-1" (Irlanda).

Disponibilidade

Zonas de Disponibilidade (AZs)

Dentro da Região "sa-east-1", a AWS possui várias AZs, por exemplo: "sa-east-1a", "sa-east-1b" e "sa-east-1c".

Para garantir que seu aplicativo permaneça online mesmo se houver uma falha em um data center, você decide distribuir seus recursos:

- Você executa seus servidores web (instâncias EC2) e bancos de dados em "sa-east-1a".

- Você também configura servidores web e bancos de dados idênticos em "sa-east-1b".

- Você usa um balanceador de carga (ELB) para distribuir o tráfego entre os servidores em "sa-east-1a" e "sa-east-1b".

O que acontece se houver uma falha?

Se a Região **"sa-east-1"** falhar completamente

Isso é extremamente raro, mas se acontecesse, seu aplicativo ficaria offline, a menos que você tivesse um plano de recuperação de desastres que envolvesse a replicação para outra Região (o que é um cenário mais complexo, mas possível).

Disponibilidade

O que acontece se houver uma falha?

Se a AZ "**sa-east-1a**" **falhar** (por exemplo, um corte de energia ou um desastre natural local)

O balanceador de carga detectaria que os servidores em "sa-east-1a" não estão respondendo e redirecionaria todo o tráfego para os servidores em "sa-east-1b", que estão em um data center fisicamente separado e ainda operacional.

Seus clientes notariam, no máximo, uma pequena interrupção ou lentidão, mas seu aplicativo continuaria funcionando.

Disponibilidade

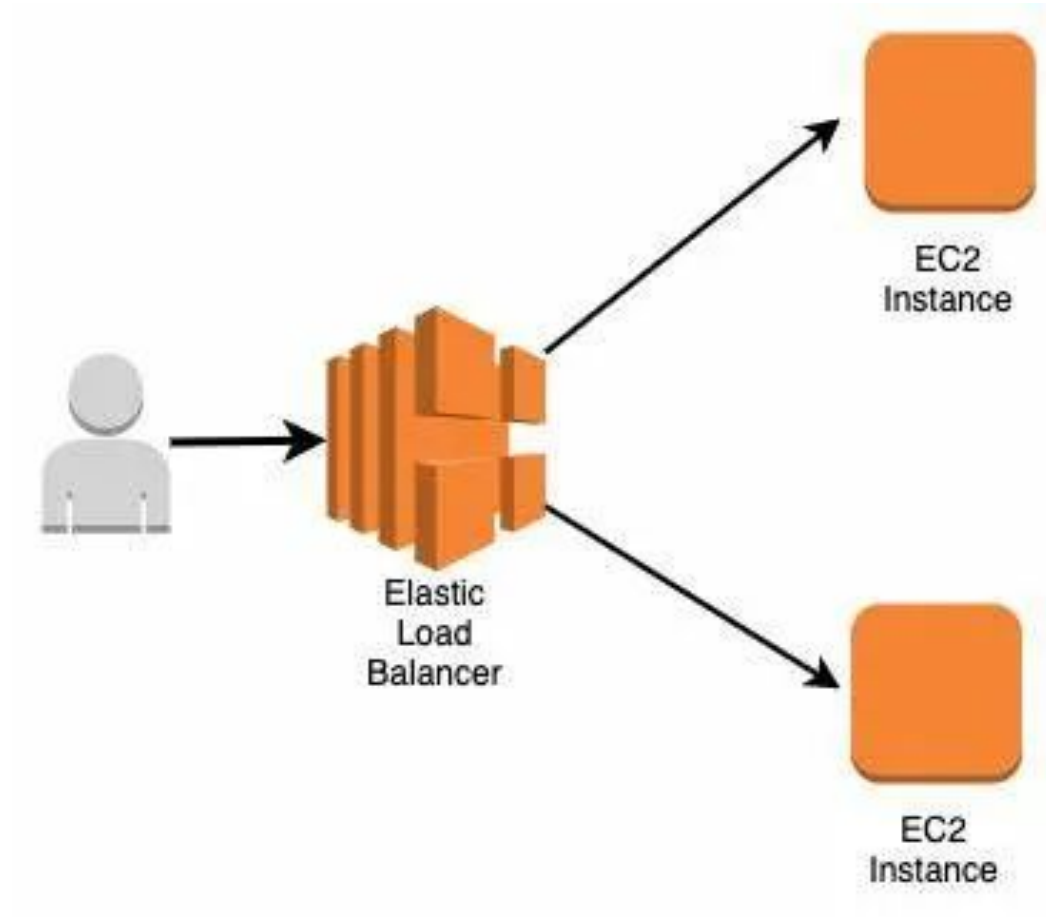
A estratégia padrão para alta disponibilidade na AWS envolve:

Implantação Multi-AZ: A aplicação é implantada em servidores (instâncias EC2) distribuídos em pelo menos duas AZs dentro de uma Região.

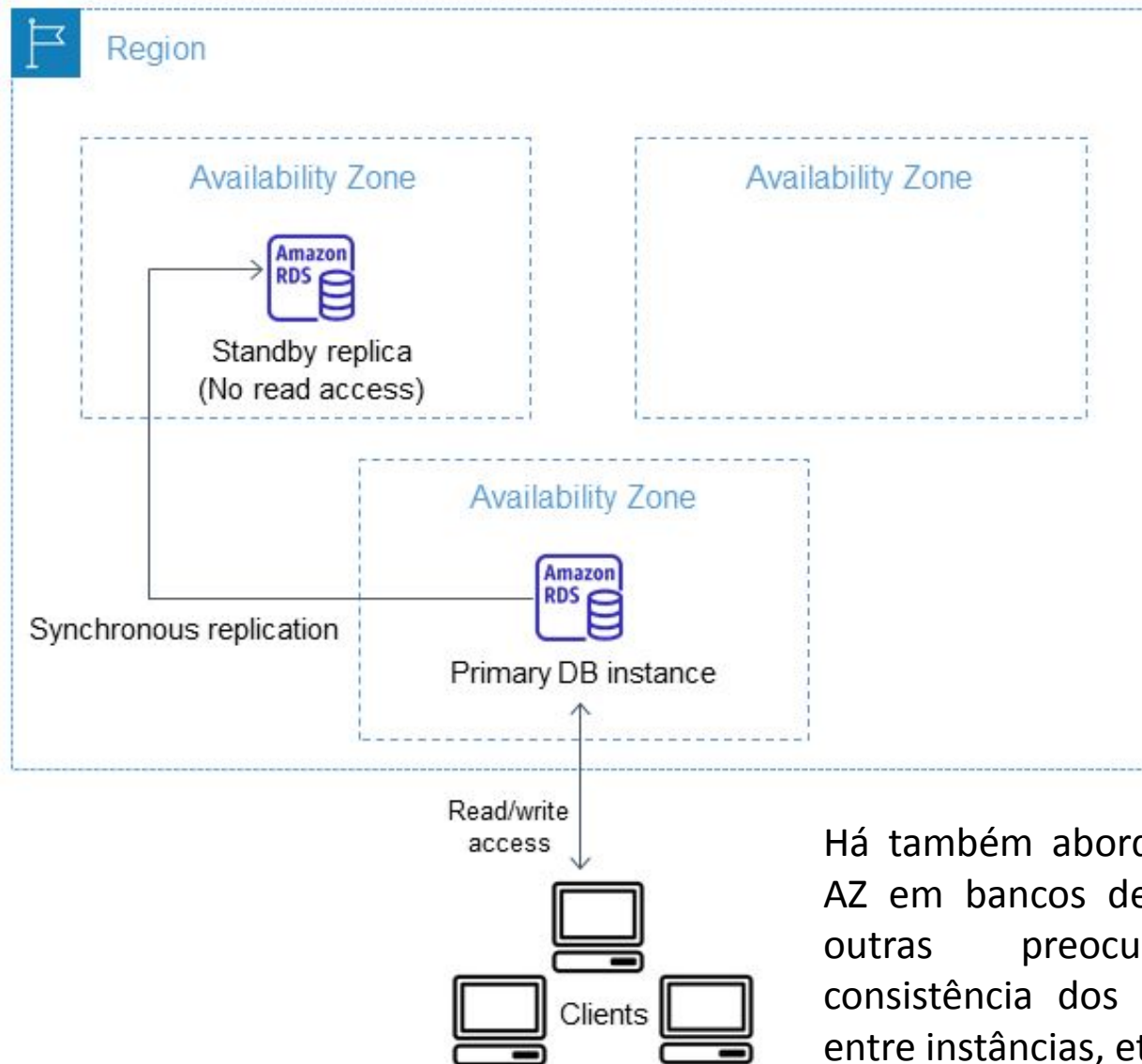
Balanceamento de Carga: Um Elastic Load Balancer (ELB) é colocado na frente das instâncias da aplicação. O ELB distribui o tráfego de entrada entre as instâncias em todas as AZs ativas.

Failover Automático: O ELB monitora continuamente a saúde das instâncias. Se uma instância ou uma AZ inteira ficar indisponível, o ELB detecta a falha e pára de enviar tráfego para ela, redirecionando automaticamente todas as solicitações para as instâncias saudáveis na outra AZ.

Disponibilidade



Disponibilidade



Há também abordagens para multi AZ em bancos de dados, mas são outras preocupações como: consistência dos dados, replicação entre instâncias, etc.

Tolerância a Falhas

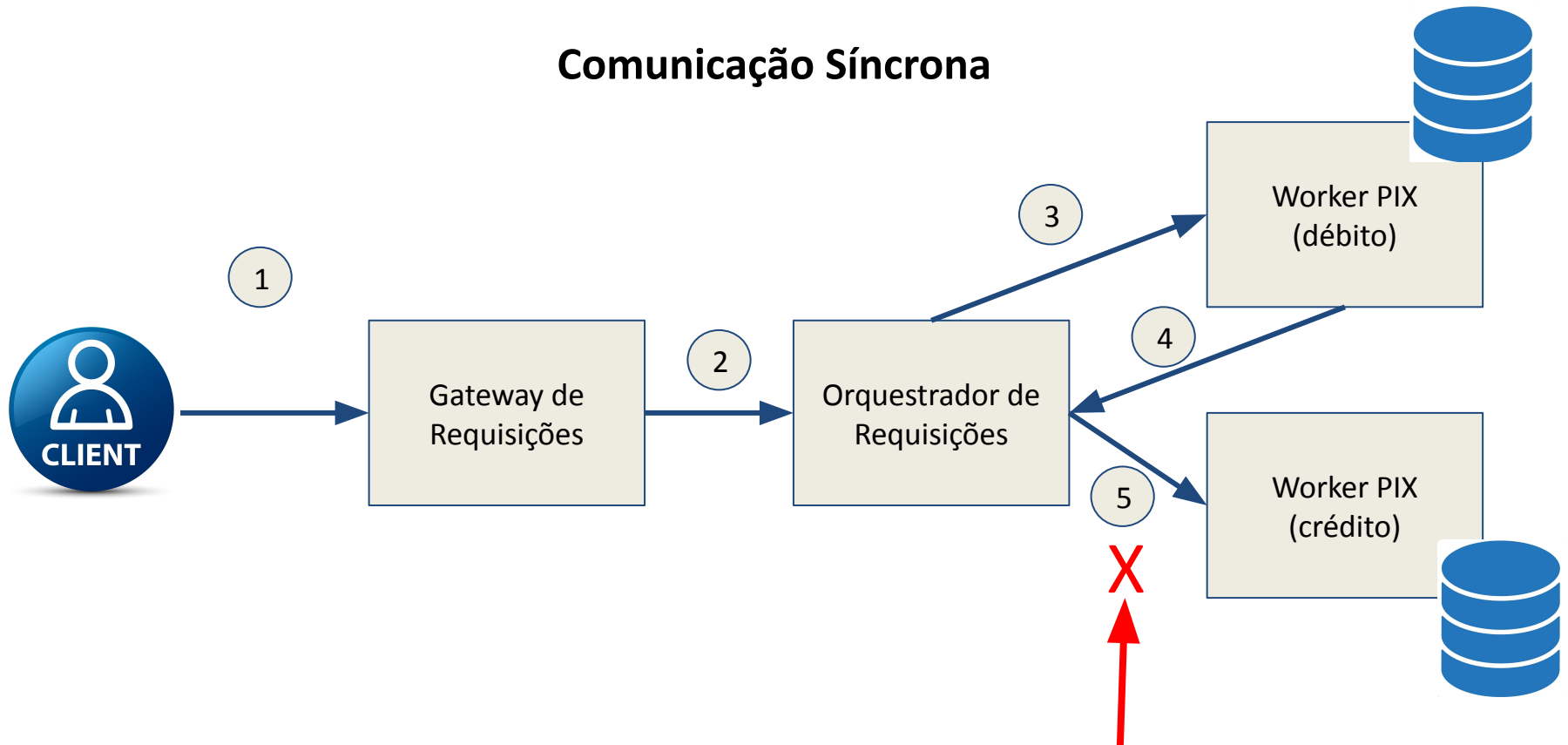
Tolerância a Falhas

A tolerância a falhas é a propriedade que permite a um sistema continuar operando corretamente, possivelmente em um nível de desempenho degradado, mesmo na presença de uma ou mais falhas em seus componentes.

É o conjunto de mecanismos que, na prática, implementam a alta disponibilidade.

Tolerância a Falhas

Comunicação Síncrona



Falhou antes de completar o crédito, mas o débito foi feito. Como proceder?

Tolerância a Falhas

Um orquestrador não pode simplesmente falhar no meio de uma transação, pois o dinheiro poderia "sumir" (ser debitado mas não creditado).

Uma solução recorrente é usar **Filas de Mensagens** (Message Queues).

O Gateway não chama o Orquestrador diretamente.

Em vez disso, ele publica uma mensagem "Processar PIX XYZ" em uma fila de mensagens (ex: Amazon SQS ou Apache Kafka), que é, por si só, um serviço distribuído e tolerante a falhas.

Tolerância a Falhas

Múltiplos "workers" do Orquestrador, rodando em diferentes AZs, "escutam" essa fila.

Um worker pega a mensagem para processar. Se esse worker falhar no meio do caminho, a mensagem não é perdida.

A fila garante que, após um tempo (timeout), a mensagem reapareça para que outro worker saudável a processe.

Isso garante que nenhuma transação seja perdida.

Tolerância a Falhas

O Banco de Dados

Redundância Extrema: Não basta um simples RDS Multi-AZ. Um sistema como o PIX usaria um banco de dados distribuído de altíssima performance e consistência (ex: Amazon Aurora Global Database, Google Spanner, ou um cluster de CockroachDB).

Os dados são replicados de forma síncrona através de múltiplas AZs. Para recuperação de desastres, os dados também são replicados de forma assíncrona para uma Região geográfica completamente diferente. Qualquer transação é escrita em múltiplos locais antes de ser confirmada.

Atomicidade

A operação "debitar da conta A e creditar na conta B" é feita dentro de uma transação de banco de dados atômica. Se qualquer parte falhar, a transação inteira é revertida (ROLLBACK), garantindo que o dinheiro nunca fique em um estado inconsistente.

Serviço de Notificação

Desacoplamento: Após o Orquestrador confirmar que a transação foi registrada com sucesso, ele não envia a notificação diretamente. Ele publica uma mensagem "PIX XYZ Concluído" em uma outra fila de mensagens.

Redundância: Múltiplos workers do serviço de notificação consomem mensagens dessa fila e enviam as confirmações. Se um notificador falhar, outro assume.

Isso evita que uma falha na notificação (menos crítica) afete o processamento de novas transações financeiras.

Recuperabilidade

Recuperabilidade

A recuperabilidade é a capacidade de um sistema de, após uma falha ou desastre, **restabelecer um nível de desempenho especificado e recuperar os dados** diretamente afetados.

Enquanto a tolerância a falhas e a disponibilidade lidam com a prevenção de interrupções, a recuperabilidade lida com a resposta e a recuperação após a ocorrência de uma interrupção significativa.

Recuperabilidade

A ascensão da computação em nuvem revolucionou a recuperação de desastres, tornando estratégias mais sofisticadas.

A capacidade de provisionar e desprovisionar infraestrutura sob demanda permite que as empresas testem seus planos de forma regular e automatizada, algo que era proibitivamente caro e arriscado com data centers físicos, aumentando drasticamente a confiança na recuperabilidade do sistema.

Recuperabilidade

O "VendeTudo" é um site de e-commerce que processa centenas de pedidos por hora. A equipe de tecnologia definiu os seguintes objetivos de recuperação para o sistema:

RTO (Recovery Time Objective / Objetivo de Tempo de Recuperação): O sistema deve voltar a operar em no máximo 30 minutos após uma falha crítica.

RPO (Recovery Point Objective / Objetivo de Ponto de Recuperação): Em caso de falha, a perda de dados de transações (pedidos, cadastros) não pode exceder 5 minutos.

Recuperabilidade

A equipe implementou as seguintes medidas no banco de dados principal (que armazena pedidos, clientes, produtos, etc.):

Backups Contínuos: O banco de dados está configurado para salvar continuamente os logs de transação. Isso permite restaurar o banco de dados para qualquer ponto específico no tempo (com granularidade de minutos) nos últimos 7 dias.

Snapshots Diários: Todo dia, à meia-noite, um "snapshot" completo (uma cópia integral) do banco de dados é criado e armazenado de forma segura e separada. Esses snapshots são mantidos por 30 dias.

Recuperação: A equipe criou documentos passo a passo que descrevem exatamente o que fazer em diferentes cenários de falha. Eles também realizam simulações de desastre a cada três meses para testar e otimizar esses procedimentos.

Segurança

A segurança é o atributo que descreve a capacidade de um sistema de proteger informações e dados contra acesso, uso, divulgação, alteração ou destruição não autorizados.

Sub-características

Confidencialidade: os dados são acessíveis apenas a usuários autorizados.

Integridade: os dados não podem ser modificados sem autorização.

Não-repúdio: as ações podem ser provadas como tendo ocorrido.

Responsabilidade: as ações podem ser rastreadas até uma entidade.

Autenticidade: a identidade de um usuário ou recurso pode ser provada.

Segurança

A segurança na nuvem, exemplificada pela AWS, opera sob um **Modelo de Responsabilidade Compartilhada** (Shared Responsibility Model).

A AWS é responsável pela segurança **da** nuvem, o que inclui a proteção da infraestrutura física global (data centers, rede, hardware).

O cliente, por sua vez, é responsável pela segurança **na** nuvem, o que engloba a configuração de seus próprios recursos, o gerenciamento de dados e o controle de acesso.

Autenticação e Autorização

O serviço AWS Identity and Access Management (IAM) permite a criação de políticas granulares que definem precisamente "quem" (usuário, grupo, serviço) pode executar "quais ações" (ex: ler, escrever, apagar) em "quais recursos" (ex: um bucket S3, uma instância EC2). A Autenticação Multifator (MFA) adiciona uma camada extra de proteção para logins.

Proteção de Rede

A Virtual Private Cloud (VPC) permite que os clientes criem redes virtuais isoladas logicamente. Dentro de uma VPC, os Security Groups atuam como firewalls no nível da instância, controlando o tráfego de entrada e saída, enquanto as Network Access Control Lists (NACLs) atuam como firewalls no nível da sub-rede.

Criptografia

A AWS facilita a criptografia de dados tanto em repouso (armazenados em serviços como S3 ou EBS) quanto em trânsito (usando TLS para comunicações de rede). O AWS Key Management Service (KMS) permite que os clientes gerenciem centralmente as chaves de criptografia.

Detecção de Ameaças e Monitoramento

Serviços como Amazon GuardDuty usam aprendizado de máquina para analisar logs e detectar atividades maliciosas ou não autorizadas, enquanto o AWS CloudTrail registra todas as chamadas de API feitas na conta, fornecendo uma trilha de auditoria completa para análise de segurança e conformidade.²⁸

Manutenibilidade

Manutenibilidade

A manutenibilidade mede a facilidade com que um sistema de software pode ser modificado para corrigir defeitos, melhorar o desempenho ou outras características, ou adaptar-se a um ambiente em mudança (evolução).

Um sistema com alta manutenibilidade permite que as equipes de desenvolvimento respondam rapidamente às novas necessidades do negócio com custo e risco reduzidos.

Sub-atributos

Modularidade, Reusabilidade, Analisabilidade e Modificabilidade.

Manutenibilidade

Os princípios arquiteturais que melhoram a manutenibilidade

Implantação Independente: Cada módulo/serviço/microserviço pode ser modificado, testado e implantado de forma independente, sem afetar o resto do sistema.

Se um bug for encontrado no serviço de pagamentos, a equipe pode corrigir e implantar apenas esse serviço, em vez de ter que reconstruir e reimplantar toda a aplicação de e-commerce. Isso acelera drasticamente os ciclos de desenvolvimento e correção.

Baixo Acoplamento: Os serviços se comunicam através de APIs bem definidas (geralmente RESTful ou gRPC), não compartilhando código ou bancos de dados.

Uma equipe pode reescrever completamente a implementação interna de um serviço em uma nova linguagem de programação, e desde que o contrato da API seja mantido, nenhum outro serviço será afetado.

Manutenibilidade

Contextos Delimitados: Cada serviço é modelado em torno de uma capacidade de negócio específica (ex: gerenciamento de inventário, perfil do cliente, processamento de pedidos). Isso torna a base de código de cada serviço menor, mais focada e mais fácil para um desenvolvedor entender e modificar com segurança.

A **manutenibilidade** em microsserviços apresenta uma troca.

A complexidade dentro de cada serviço é reduzida, mas a complexidade geral do sistema — a rede de interações entre os serviços — aumenta.

A **depuração** de uma solicitação que **atravessa múltiplos serviços** torna-se um desafio significativo. Consequentemente, a manutenibilidade em uma arquitetura de microsserviços depende criticamente de uma infraestrutura de DevOps madura, incluindo automação de implantação, testes de integração robustos e, crucialmente, sistemas de observabilidade para rastrear o comportamento do sistema distribuído.

Observabilidade

Observabilidade

A observabilidade é a capacidade de medir o estado interno de um sistema analisando os dados que ele gera, como logs, métricas e rastreamentos (traces).

Enquanto o monitoramento tradicional tende a responder a perguntas pré-definidas ("Qual é o uso da CPU?"), a observabilidade permite explorar e fazer novas perguntas para entender comportamentos inesperados ("Por que as solicitações para usuários no Brasil estão mais lentas nos últimos 15 minutos?").

Em arquiteturas de microsserviços complexas como a da Netflix, onde uma única solicitação do usuário pode desencadear uma cascata de chamadas através de centenas de serviços, a depuração tradicional é impossível.

A observabilidade torna-se um atributo de qualidade não negociável para entender o comportamento do sistema, diagnosticar problemas e otimizar o desempenho.

Construída sobre **três pilares de dados**:

Logs: São registros de eventos imutáveis e com timestamp. Eles fornecem o contexto mais detalhado sobre um evento específico, como uma mensagem de erro com um stack trace. Exemplo: INFO: User '123' logged in successfully from IP 'x.x.x.x' at 2023-10-27T10:00:00Z.40

Métricas: São agregações numéricas de dados ao longo do tempo, otimizadas para armazenamento e consulta. Elas são ideais para dashboards e alertas. Exemplo: A contagem de logins bem-sucedidos por minuto, a latência média de resposta da API ou a taxa de erro de 5xx.

Construída sobre **três pilares de dados**:

Rastreamentos (Distributed Traces): Mostram o ciclo de vida de uma solicitação à medida que ela se propaga por múltiplos serviços. Cada etapa da jornada é capturada como um "span", e o conjunto de spans para uma solicitação forma um "trace". Isso é inestimável para identificar gargalos de latência e entender as dependências entre serviços.

Por exemplo, ao analisar dados de rastreamento, eles podem identificar o serviço mais lento em uma cadeia de chamadas que determina a latência geral e otimizar a utilização de recursos.

Essa visibilidade permite que as equipes negociem Service Level Agreements (SLAs) de forma mais eficaz e equilibrem o custo e a latência, resultando em economias de milhões de dólares.

Usabilidade

Usabilidade

A usabilidade refere-se ao grau em que um sistema pode ser usado por usuários específicos para atingir objetivos definidos com eficácia, eficiência e satisfação em um contexto de uso específico.

Um sistema com alta usabilidade é intuitivo, fácil de aprender e livre de frustrações, permitindo que os usuários se concentrem em suas tarefas em vez de lutar contra a interface.

A Apple é renomada por priorizar a usabilidade, e seu ecossistema iOS é um exemplo de como os princípios de usabilidade podem ser aplicados em escala.

A empresa estabelece e impõe um conjunto rigoroso de diretrizes através de suas Human Interface Guidelines (HIG), que garantem um alto grau de consistência e intuitividade em todos os aplicativos da plataforma.

Consistência

Este é talvez o princípio mais importante. As HIG definem padrões para navegação, ícones, gestos e controles de interface. Ao aderir a esses padrões, os desenvolvedores criam aplicativos que parecem familiares aos usuários desde o primeiro uso, reduzindo a carga cognitiva e acelerando a curva de aprendizado.

Feedback Direto

Cada interação do usuário recebe uma resposta imediata. Tocar em um botão causa uma mudança visual sutil; ações importantes podem ser acompanhadas por um feedback tátil.

Isso confirma ao usuário que o sistema registrou sua ação, criando uma sensação de controle e capacidade de resposta.

Reconhecimento sobre Recordação

As interfaces gráficas são projetadas para que os usuários possam reconhecer ações e opções visualmente, em vez de terem que se lembrar de comandos complexos. Menus, ícones e rótulos claros servem como pistas, guiando o usuário através das funcionalidades do aplicativo.

Design Ergonômico

As diretrizes levam em consideração a forma como as pessoas seguram e interagem com os dispositivos. Por exemplo, os alvos de toque (botões e outros elementos interativos) devem ter um tamanho mínimo de 44x44 pixels para serem facilmente pressionados com um dedo sem erros.

Portabilidade

Portabilidade

A portabilidade é a facilidade com que um sistema de software pode ser transferido de um ambiente de computação para outro.

Os ambientes podem variar em termos de sistema operacional, hardware ou provedor de nuvem.

Alta portabilidade reduz a dependência de um único fornecedor (vendor lock-in) e aumenta a flexibilidade da implantação.

Aplicações em Contêineres com Docker e Kubernetes

A tecnologia de contêineres, popularizada pelo Docker, revolucionou a portabilidade do software.

Antes dos contêineres, a portabilidade era um desafio significativo. As aplicações dependiam de bibliotecas, runtimes e configurações específicas do sistema operacional hospedeiro, o que levava ao clássico problema "funciona na minha máquina, mas não na produção".

Aplicações em Contêineres com Docker e Kubernetes

Um contêiner resolve isso empacotando uma aplicação com todas as suas dependências — código, runtime (ex: JVM, Node.js), bibliotecas do sistema, variáveis de ambiente e arquivos de configuração — em uma única unidade padronizada e executável chamada imagem de contêiner.

Esta imagem pode ser executada em qualquer máquina que tenha um runtime de contêiner (como o Docker Engine) instalado, independentemente do sistema operacional subjacente (Linux, Windows ou macOS).

Docker é a tecnologia que empacota sua aplicação e todas as suas dependências em unidades isoladas e portáteis (contêineres).

Kubernetes é a plataforma que automatiza a implantação, o dimensionamento e o gerenciamento de aplicações containerizadas.

Você cria suas aplicações em contêineres usando Docker.

Você gerencia e executa esses contêineres em escala usando Kubernetes.

Exemplo Banco | Microserviços:

Serviço de Autenticação: Lida com login, senha e biometria.

Serviço de Contas: Gerencia saldos e extratos.

Serviço de Transferências (PIX/TED): Processa o envio e recebimento de dinheiro.

Serviço de Cartões: Gerencia o limite, a fatura e o bloqueio de cartões.

Serviço de Notificações: Envia e-mails e push notifications.

API Gateway: Ponto de entrada único que direciona as requisições do aplicativo móvel para o serviço correto.

O **Docker** é usado para "**empacotar**" cada um desses microserviços em unidades padronizadas e seguras chamadas contêineres.

Garantir que cada serviço rode em um ambiente idêntico, isolado e seguro, não importando onde ele seja executado.

Já o **Kubernetes**, foca em **automatizar** a implantação, a escalabilidade e a recuperação de falhas de todos os microserviços, garantindo que a plataforma do banco esteja sempre online e performática.

Extensibilidade

Extensibilidade

A extensibilidade é a qualidade de um software que considera futuras ampliações. Um sistema extensível permite a adição de novas funcionalidades com o mínimo de modificações em seu código ou arquitetura existente.

Extensibilidade

A extensibilidade é a chave para a criação de um ecossistema próspero em torno de um produto.

Ao permitir que outros construam sobre sua plataforma, o ecossistema cria um poderoso efeito de rede:

mais aplicativos atraem mais clientes, e mais clientes atraem mais desenvolvedores.

A arquitetura de uma plataforma extensível deve ser projetada com um "núcleo limpo" e pontos de extensão bem definidos, onde a estabilidade e a retrocompatibilidade dessas interfaces se tornam de importância crítica.

Widgets na Tela de Início (iOS 14+)

Problema:

Usuários querem ver informações rápidas de seus aplicativos favoritos sem abri-los. A tela de início padrão da Apple é relativamente estática.

Abordagem **SEM Extensibilidade**:

A Apple teria que codificar widgets para cada um de seus aplicativos e para os aplicativos de terceiros mais populares, ou os desenvolvedores teriam que encontrar soluções complexas para exibir informações na tela de início.

Widgets na Tela de Início (iOS 14+)

Abordagem **COM Extensibilidade**:

A Apple forneceu o WidgetKit framework (um conjunto de APIs e ferramentas) para desenvolvedores.

Desenvolvedores de aplicativos (como Spotify, Twitter, Clima, etc.) podem usar o WidgetKit para criar seus próprios widgets personalizados que exibem dados relevantes do aplicativo.

Resultado

Milhões de desenvolvedores puderam adicionar uma nova e rica funcionalidade aos seus aplicativos, estendendo a utilidade da tela de início do iOS, tudo sem que a Apple tivesse que reescrever seus próprios sistemas para cada app. O sistema operacional foi estendido de forma massiva e versátil.

Tabela de Trade-offs

Atributo	Desempenho	Segurança	Manutenibilidade	Escalabilidade	Usabilidade
Desempenho		-	-	+	+
Segurança	-		-	O	-
Manutenibilidade	-	-		+	O
Escalabilidade	+	O	+		O
Usabilidade	+	-	O	O	

Legenda:

+ (Relação geralmente positiva/sinérgica)

- (Relação geralmente negativa/confitante)

O (Relação geralmente neutra ou contextual)

Exercício Prático

Exemplo

Você faz parte de uma equipe responsável por projetar a arquitetura de um sistema de hub de jogos, similar a Steam ou Omen Gaming Hub.

Esse sistema permitirá que usuários se cadastrem, comprem, façam o download e joguem através da plataforma, além de oferecer recursos como chat e compartilhamento de conquistas.

O sistema precisa ser escalável, oferecer uma experiência de jogo sem lag, ser seguro e permitir integração com diferentes plataformas (consoles, PC, mobile).

Exercício prático

O seu objetivo é refletir e documentar sobre os atributos de qualidade de um sistema e julgar quais os mais importantes, fornecendo um balanceamento entre eles para o tipo de sistema sendo modelado.

Identifique e descreva os principais atributos de qualidade que são críticos para o sistema de *hub de jogos*. Exemplos:

- Desempenho (especialmente em relação à baixa latência para jogos online)
- Segurança (proteção de dados do usuário e contra ataques DDoS)

RESPONDA

- Quais atributos de qualidade são essenciais para o sucesso do sistema?
- Como a arquitetura do sistema pode influenciar diretamente esses atributos?

Exercício prático

O que deve ser entregue?

- **Relatório**
 - Atributos de qualidade identificados e discutidos, pontos fortes e fracos de cada um deles
 - Proposta de 5 decisões arquiteturais que você tomaria para garantir que o sistema atenda a esses requisitos
 - Justificativas das decisões e impactos de cada decisão
- **Apresentação** do relatório para toda a turma

Para cada decisão arquitetural, responda às seguintes perguntas:

- Qual é a decisão?
- Quais atributos de qualidade ela impacta?
 - Identifique qual atributo (ou atributos) de qualidade essa decisão suporta e explique o motivo.
- Quais trade-offs estão envolvidos?
 - Discuta os possíveis impactos negativos dessa decisão.
 - Como ela pode afetar outros atributos de qualidade?

Exemplo

Decisão Arquitetural 1: Adotar uma arquitetura de Microsserviços para as funcionalidades de negócio e sociais, como Loja, Inventário de Jogos, Perfis de Usuário, Chat e Conquistas.

Quais atributos de qualidade ela impacta?

Principalmente: Escalabilidade e Disponibilidade. Secundariamente: Manutenibilidade.

Decisão Arquitetural 2: Utilizar um banco de dados relacional (SQL) para transações críticas (Pagamentos, Contas de Usuário) e bancos de dados NoSQL para dados sociais e de alta volatilidade (Chats, Status Online, Feeds de Atividade).

Quais atributos de qualidade ela impacta?

Principalmente: Segurança (Consistência) e Escalabilidade/Desempenho.

Disserte sobre os atributos e o balanceamento ideal, fornecendo pontos positivos e negativos.

Sorteio dos temas entre as equipes, deve-se escolher (ou simular) um sistema dentro da categoria.

Temas:

- [1x] Gamming hub (Steam, OMEM Gamming Hub)
- [1x] Pagamentos online
- [1x] PIX
- [1x] Streaming de vídeo (Netflix, Prime, Disney+)
- [1x] Plataforma de Educação à Distância (Alura, Udemy)
- [1x] Redes sociais (Instagram, Tiktok, X)
- [1x] Transporte urbano (Uber, 99Taxi)
- [1x] Pedidos de comida e bebidas (iFood, Ze Delivery, Uber Eats)
- [2x] Desenvolvimento social local - simular sistema (sistema ligação de produtores locais com consumidores, vendas por plataforma de produtos locais, etc) - Validar o tema com o professor.

Dúvidas?



www.shutterstock.com · 744867163